

DOCUMENTAÇÃO TÉCNICA

Aplicação de Comunicação Aumentativa e Alternativa (CAA)

Java + Spring Boot + React + PostgreSQL

Versão: 1.0 Data: 12/08/2026

Documento de arquitetura, produto, MVP e roadmap técnico.

1. Visão geral do projeto

O projeto consiste em uma aplicação de comunicação voltada a crianças com dificuldade na fala. A interface será composta por botões grandes, visuais e simples. Ao selecionar um botão, a aplicação reproduzirá em voz alta uma frase previamente associada àquela intenção.

Exemplo: a criança seleciona “Fome” e o dispositivo reproduz “Estou com fome.”

A proposta inicial é manter a experiência extremamente simples e acessível, evitando digitação e reduzindo a quantidade de decisões necessárias para comunicar uma necessidade.

Conceito: o projeto se enquadra no contexto de Comunicação Aumentativa e Alternativa (CAA/AAC), mas esta documentação descreve uma solução de software e não substitui avaliação ou orientação de profissionais especializados.

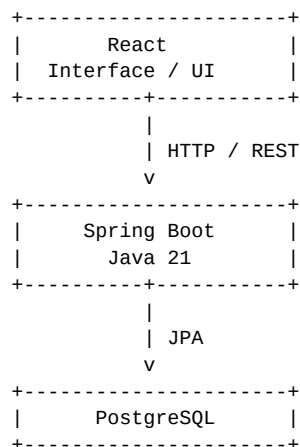
1.1 Objetivos

- Criar uma interface simples e acessível para comunicação por seleção visual.
- Permitir que o usuário comunique necessidades, emoções, ações e pessoas por meio de botões.
- Converter a intenção selecionada em fala usando Text-to-Speech.
- Construir uma arquitetura Java + React adequada para evolução futura.
- Usar o projeto como aplicação prática de Java, Spring Boot, React, banco de dados, testes, Docker e posteriormente AWS.

1.2 Princípios de arquitetura

- Simplicidade: evitar complexidade desnecessária no MVP.
- Separação de responsabilidades: frontend cuida da experiência; backend cuida dos dados e regras.
- Evolução incremental: começar pequeno e adicionar recursos somente quando houver necessidade.
- Acessibilidade: tamanho, contraste, navegação e feedback devem ser considerados desde o início.
- Privacidade: no MVP, evitar dados pessoais da criança.

2. Arquitetura proposta



Text-to-Speech:

React -> Web Speech API -> Áudio no dispositivo

No MVP, o Text-to-Speech será executado preferencialmente no próprio frontend por meio da API de voz disponível no navegador. Isso evita introduzir um serviço externo de voz antes de existir necessidade real.

2.1 Stack tecnológica

Camada	Tecnologia	Responsabilidade
Frontend	React + TypeScript	Interface e interação
Build frontend	Vite	Execução e build
Backend	Java 21 + Spring Boot	API e regras de negócio
API	REST/JSON	Comunicação React ↔ Java
Persistência	PostgreSQL	Armazenamento
ORM	Spring Data JPA	Persistência de entidades
Build backend	Maven	Dependências e build
Testes	JUnit + Mockito	Testes unitários
Testes frontend	Vitest + Testing Library	Testes de componentes
Containers	Docker + Docker Compose	Ambiente local
Cloud futura	AWS	Hospedagem e serviços gerenciados
Voz	Web Speech API	Text-to-Speech inicial

2.2 Responsabilidades

Componente	Responsabilidade
React	Exibir categorias, botões, estados da interface e acionar a fala.
Spring Boot	Fornecer categorias/botões, validar dados e concentrar regras de negócio.
PostgreSQL	Persistir categorias e configurações dos botões.
Web Speech API	Converter o texto do botão em fala no dispositivo.

3. MVP — Minimum Viable Product

O MVP deve provar uma única hipótese principal: uma criança consegue selecionar uma intenção visualmente e o aplicativo consegue transformá-la em uma fala compreensível com poucos toques.

3.1 Escopo do MVP

- Uma tela principal.
- Categorias básicas.
- Botões grandes com ícone e texto.
- Frases associadas a cada botão.
- Reprodução de voz ao tocar no botão.
- API REST para carregar categorias e botões.
- Banco PostgreSQL.

- Docker Compose para ambiente local.
- Dados iniciais (seed) para a aplicação funcionar imediatamente.

3.2 Fora do MVP

- Login e autenticação.
- Cadastro de crianças.
- Gravação de voz.
- Inteligência artificial.
- Reconhecimento de fala.
- Microserviços.
- Kubernetes.
- Integração com AWS Polly.
- Analytics detalhado.
- Sincronização entre vários dispositivos.

3.3 Critério de sucesso

O MVP pode ser considerado funcional quando um usuário abrir a aplicação, visualizar os botões, tocar em um botão e ouvir a frase correspondente sem precisar digitar nada.

4. Modelo funcional

4.1 Categorias iniciais

Categoria	Exemplos
Necessidades	Fome, sede, banheiro, sono, dor, ajuda
Emoções	Feliz, triste, bravo, medo, cansado
Pessoas	Mamãe, papai, professor, amigo
Ações	Quero, não quero, brincar, parar, ir
Saudações	Bom dia, boa tarde, boa noite, obrigado

4.2 Exemplos de botões

Label	Frase falada	Categoria
Bom dia	Bom dia!	Saudações
Fome	Estou com fome.	Necessidades
Sede	Estou com sede.	Necessidades
Banheiro	Preciso ir ao banheiro.	Necessidades
Ajuda	Preciso de ajuda.	Necessidades
Feliz	Estou feliz.	Emoções
Triste	Estou triste.	Emoções
Mamãe	Mamãe!	Pessoas

Label	Frase falada	Categoria
Parar	Quero parar.	Ações
Não quero	Eu não quero.	Ações

4.3 Ideia de layout



5. Backend — Spring Boot

5.1 Estrutura de pacotes sugerida

```
src/main/java/com/seuprojeto
■■■ controller
■   ■■■ CategoryController.java
■   ■■■ CommunicationButtonController.java
■■■ service
■   ■■■ CategoryService.java
■   ■■■ CommunicationButtonService.java
■■■ repository
■   ■■■ CategoryRepository.java
■   ■■■ CommunicationButtonRepository.java
■■■ entity
■   ■■■ Category.java
■   ■■■ CommunicationButton.java
■■■ dto
■   ■■■ CategoryResponse.java
■   ■■■ CommunicationButtonResponse.java
■■■ config
■   ■■■ DataInitializer.java
```

5.2 Endpoints iniciais

Método	Endpoint	Objetivo
GET	/api/categories	Listar categorias
GET	/api/categories/{id}	Consultar categoria
GET	/api/buttons	Listar botões ativos
GET	/api/buttons/{id}	Consultar botão

5.3 Exemplo de resposta

```
GET /api/buttons
```

```
[
```

```
{
  "id": 1,
  "label": "Fome",
  "speechText": "Estou com fome.",
  "icon": "food",
  "categoryId": 1,
  "order": 1
}
```

6. Banco de dados

6.1 Modelo inicial

```
CATEGORY
-----
id PK
name
icon
display_order
active

COMMUNICATION_BUTTON
-----
id PK
category_id FK -> CATEGORY.id
label
speech_text
icon
display_order
active
```

6.2 Regras

- Uma categoria pode possuir vários botões.
- Um botão pertence a uma única categoria no modelo inicial.
- Botões inativos não devem aparecer para o usuário.
- display_order controla a posição de apresentação.
- speech_text é o texto utilizado pelo mecanismo de voz.

7. Frontend — React

7.1 Estrutura sugerida

```
src/
  components/
    CommunicationButton/
    CategoryCard/
    Header/
  pages/
    Home/
  services/
    api/
  hooks/
  types/
  assets/
  App.tsx
```

7.2 Fluxo de interação

1. React carrega a tela.
2. Frontend chama GET /api/categories e/ou GET /api/buttons.
3. Backend retorna os dados.
4. React renderiza os botões.
5. Criança toca em "Fome".

6. React obtém `speechText = "Estou com fome."`
7. Web Speech API reproduz a frase.

Exemplo conceitual de Text-to-Speech:

```
const utterance = new SpeechSynthesisUtterance(
  "Estou com fome."
);

speechSynthesis.speak(utterance);
```

8. Acessibilidade e UX

A acessibilidade não deve ser tratada como etapa posterior. A interface foi concebida para um usuário que pode ter dificuldade de fala e também pode apresentar diferentes necessidades motoras, cognitivas ou sensoriais.

- Botões grandes e fáceis de tocar.
- Texto curto e objetivo.
- Ícones consistentes e fáceis de reconhecer.
- Contraste adequado entre texto e fundo.
- Poucos elementos por tela.
- Feedback visual após o toque.
- Possibilidade futura de ajustar tamanho dos botões.
- Possibilidade futura de configurar velocidade e voz.
- Evitar depender exclusivamente de cor para transmitir significado.

Nota: decisões de acessibilidade devem ser validadas com usuários reais e, quando possível, com profissionais especializados.

9. Privacidade e segurança

Como o público-alvo envolve crianças, a primeira versão deve adotar uma postura conservadora de coleta de dados.

9.1 MVP

- Não armazenar nome da criança.
- Não armazenar fotos.
- Não armazenar gravações de voz.
- Não coletar localização.
- Não criar perfil pessoal sem necessidade.
- Evitar analytics identificável.

9.2 Evolução

Caso no futuro sejam necessários perfis, personalização ou sincronização, a arquitetura deverá ser revisada para tratar autenticação, autorização, minimização de dados, retenção, consentimento e requisitos legais aplicáveis.

10. Docker e ambiente local

```

docker-compose.yml

services:

  postgres:
    image: postgres
    environment:
      POSTGRES_DB: comunicacao
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"

  backend:
    build: ./backend
    ports:
      - "8080:8080"
    depends_on:
      - postgres

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend

```

O arquivo acima é uma representação inicial. No desenvolvimento real, as versões das imagens, variáveis de ambiente, health checks e configurações de produção deverão ser definidos explicitamente.

11. Roadmap de desenvolvimento

Fase	Entrega	Objetivo
0	Definição do produto	Fechar escopo do MVP
1	Backend básico	Criar API e persistência
2	Frontend básico	Criar tela e componentes
3	Text-to-Speech	Fazer botão falar
4	Integração	React consumindo API
5	Docker	Subir stack completa localmente
6	Testes	Cobrir regras e componentes principais
7	Acessibilidade	Melhorar UX e validar interface
8	Deploy	Publicar primeira versão

12. Backlog inicial

ID	História	Prioridade
US-01	Como usuário, quero visualizar categorias.	Alta
US-02	Como usuário, quero visualizar botões grandes.	Alta
US-03	Como usuário, quero tocar em um botão e ouvir uma frase.	Alta
US-04	Como sistema, quero carregar botões pela API.	Alta
US-05	Como sistema, quero persistir botões no PostgreSQL.	Alta

ID	História	Prioridade
US-06	Como usuário, quero identificar o botão por texto e ícone.	Alta
US-07	Como usuário, quero receber feedback visual ao tocar.	Média
US-08	Como responsável, quero personalizar botões.	Futura
US-09	Como usuário, quero montar frases.	Futura

13. Como preencher e evoluir o MVP

A ideia é que o MVP seja preenchido com dados reais de uso, e não apenas com dezenas de funcionalidades. Para cada nova ideia, deve-se perguntar: isso ajuda diretamente a criança a comunicar algo com menos esforço?

13.1 Formulário conceitual de novo botão

Campo	Exemplo
Nome curto	Fome
Texto falado	Estou com fome.
Categoria	Necessidades
Ícone	🍽️
Ordem	1
Ativo	Sim

13.2 Como decidir novos botões

- Começar pelas necessidades mais frequentes.
- Preferir frases que representem uma intenção completa.
- Evitar criar muitos botões semelhantes.
- Observar quais botões são usados com maior frequência.
- Permitir personalização posteriormente.
- Testar a compreensão dos ícones com usuários e responsáveis.

13.3 Evolução para construção de frases

[EU] + [QUERO] + [ÁGUA]

↓

"Eu quero água."

Outros exemplos:

[EU] + [NÃO QUERO] + [COMER]

[EU] + [QUERO] + [BRINCAR]

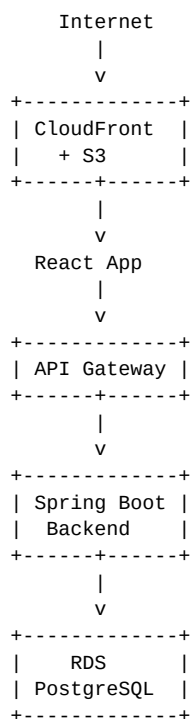
[EU] + [ESTOU] + [TRISTE]

Essa evolução transforma o aplicativo de um conjunto de frases prontas em uma ferramenta de construção de comunicação.

14. Evoluções futuras

Versão	Funcionalidade
V1	Botões e Text-to-Speech
V2	Categorias e favoritos
V3	Personalização de frases e botões
V4	Construção de frases
V5	Perfis e configurações do responsável
V6	Sincronização e múltiplos dispositivos
V7	Serviço de voz dedicado, se necessário
V8	Recursos inteligentes/IA, somente se houver benefício claro

15. Possível arquitetura AWS futura



Text-to-Speech:
React -> Web Speech API
ou, futuramente:
Backend -> serviço de voz dedicado

Essa arquitetura é uma direção futura, não uma obrigação do MVP. O objetivo é primeiro validar o produto e depois decidir quais componentes realmente precisam ser gerenciados pela AWS.

16. Critérios técnicos de qualidade

- API documentada.
- DTOs separados das entidades quando fizer sentido.
- Validação de entradas.
- Tratamento consistente de erros.

- Logs sem dados sensíveis.
- Testes unitários para regras de negócio.
- Testes de integração para endpoints importantes.
- Frontend com componentes reutilizáveis.
- Configurações por variáveis de ambiente.
- Nenhuma senha ou segredo versionado no Git.
- Docker Compose funcional para desenvolvimento.

17. Plano de implementação recomendado

Sprint 1 — Fundação

- Criar repositório Git.
- Criar projeto Spring Boot.
- Configurar PostgreSQL.
- Criar entidades Category e CommunicationButton.
- Criar repositories.
- Criar services.
- Criar endpoints GET.
- Inserir dados iniciais.

Sprint 2 — React

- Criar projeto React + TypeScript + Vite.
- Criar página Home.
- Criar componente CommunicationButton.
- Criar layout responsivo.
- Consumir API do backend.
- Exibir categorias e botões.

Sprint 3 — Voz

- Integrar Web Speech API.
- Associar speechText ao clique.
- Adicionar feedback visual.
- Testar diferentes vozes e velocidades.

Sprint 4 — Qualidade

- Criar testes backend.
- Criar testes frontend.
- Adicionar tratamento de erros.
- Criar Dockerfiles.

- Criar Docker Compose.
- Documentar execução local.

18. Definição de pronto — Definition of Done

- Funcionalidade implementada.
- Código versionado.
- Testes relevantes executados.
- API funcionando.
- Frontend integrado.
- Nenhuma credencial no repositório.
- Comportamento validado manualmente.
- Documentação atualizada quando houver mudança arquitetural.

19. Decisões arquiteturais iniciais

Decisão	Justificativa
Monólito Spring Boot	Menor complexidade para o estágio inicial.
REST	Simples, conhecido e suficiente para o MVP.
PostgreSQL	Banco relacional maduro e adequado ao domínio.
React + TypeScript	Boa separação de frontend e facilidade de evolução.
Web Speech API	Evita custo e infraestrutura de voz no MVP.
Docker Compose	Reproduz ambiente local com poucos comandos.
Sem autenticação no MVP	Não há necessidade de identidade para validar a hipótese inicial.
Sem microsserviços	Não existe benefício suficiente para justificar a complexidade.

20. Próximo passo do projeto

A próxima etapa recomendada é transformar esta documentação em uma especificação executável. A sequência ideal é:

1. Fechar a lista de categorias e os primeiros 20–30 botões.
2. Criar o modelo ER do banco.
3. Criar o projeto Spring Boot.
4. Implementar a API GET.
5. Criar o React.
6. Construir a primeira tela.
7. Integrar Text-to-Speech.
8. Colocar tudo em Docker.
9. Criar testes.
10. Fazer uma primeira validação de usabilidade.

Conclusão: o projeto deve começar como uma aplicação pequena, funcional e acessível. A arquitetura foi propositalmente desenhada para permitir evolução para personalização, construção de frases, perfis, cloud e outros recursos sem exigir que essas complexidades sejam introduzidas antes da hora.